REPORTE TAREA 1
ALGORITMOS Y COMPLEJIDAD

«Más allá de la notación asintótica: Análisis experimental de algoritmos de ordenamiento y multiplicación de matrices.»

Arturo Almonacid

9 de abril de 2026

21:28

1. Entorno experimental

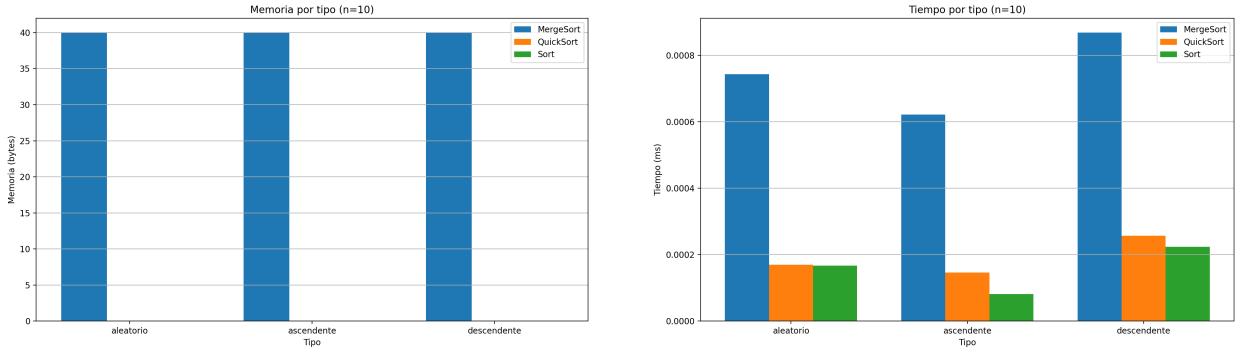
El hardware utilizado para la realización de los experimentos corresponde a un computador de escritorio equipado con un procesador AMD Ryzen™ 7 5700X, el cual cuenta con 8 núcleos y 16 hilos, y una frecuencia base de 3.4 GHz. Además, el sistema dispone de 32 GB de memoria RAM DDR4 a 3200 MHz en configuración quad channel.

En cuanto al software, se utilizó el sistema operativo Nyarch Linux de 64 bits. El código fue compilado utilizando el compilador GCC versión 15.2.1, empleando el estándar C++17 y las banderas de compilación -O2 -Wall.

Respecto a los datasets, se utilizaron aquellos que vienen del archivo generador de la tarea. No obstante, por restricciones de tiempo de ejecución, no se consideraron los arreglos de tamaño 10^7 ni las matrices de 1024×1024 debido a su elevado costo computacional.

2. Resultados y Análisis

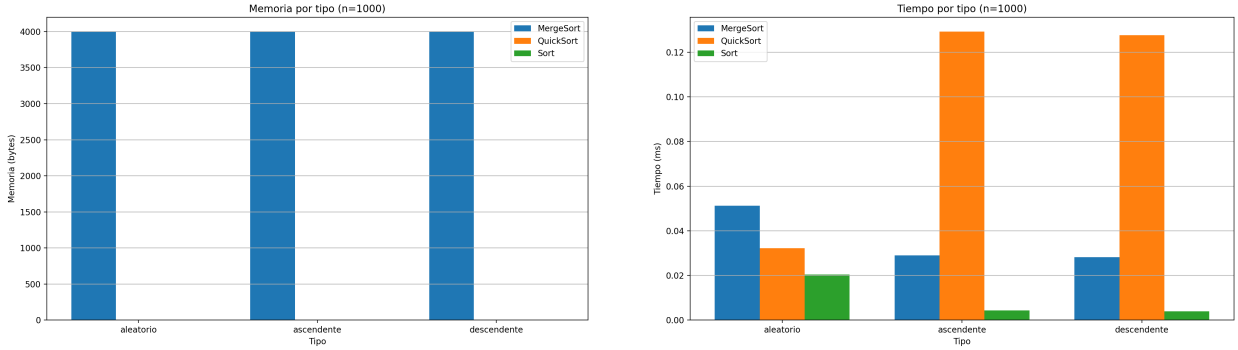
Antes de analizar los resultados, es importante destacar la metodología utilizada para la medición de memoria. En este trabajo, se consideró únicamente el peak de memoria dinámica (heap) utilizado por cada algoritmo. No se considero el stack, ya que este es memoria propia de trabajar con arreglos y no de los algoritmos, Por esta razón, se decidió excluirlo del análisis. En el caso de sort, el algoritmo no debería requerir memoria dinámica adicional. Sin embargo, en las mediciones realizadas se observó consistentemente un uso de memoria heap equivalente a 4n bytes, es decir, exactamente el tamaño de un arreglo de n enteros. Además, al modificar la función para que no retornara el vector y fuera de tipo void, dicha memoria adicional desapareció. Esto indica que el consumo registrado no proviene del algoritmo en sí, sino de la copia asociada al valor de retorno de la función utilizada en la implementación experimental. Por esta razón, dicha memoria fue descontada del análisis, sin modificar la implementación original entregada en el enunciado de la tarea.
A continuación, se presenta el análisis de los algoritmos de ordenamiento de arreglos. En particular, se examinan las gráficas obtenidas a partir de los experimentos, con el objetivo de analizar el comportamiento temporal y de memoria de cada algoritmo de manera individual.



(a) Memoria (b) Tiempos

Figura 1: Resultados para n=10^1

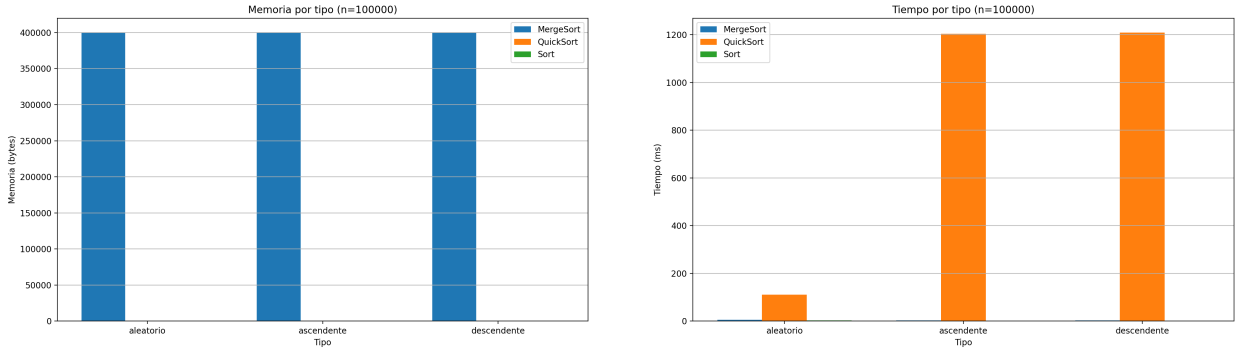
Para $n = 10^1$, MergeSort utiliza cerca de $4n$ bytes de memoria auxiliar, mientras que QuickSort y `Sort` no presentan uso significativo de heap. En tiempo de ejecución, MergeSort resulta ser el más lento en los tres casos, debido al costo de copiar y mezclar subarreglos. En cambio, QuickSort y `Sort` muestran tiempos menores y similares entre sí, con mejor desempeño en el caso ascendente y un leve aumento en el descendente, lo que puede explicarse por la forma en que la elección del pivote afecta el balance de las particiones.



(a) Memoria (b) Tiempos

Figura 2: Resultados para n=10^3

Para $n = 10^3$, MergeSort utiliza aproximadamente $4n$ bytes de memoria auxiliar, mientras que QuickSort y `Sort` no presentan uso significativo de heap. En tiempo de ejecución, MergeSort mantiene un comportamiento estable en todos los casos. QuickSort, en cambio, presenta un buen desempeño en el caso promedio, pero se degrada fuertemente en entradas ascendentes y descendentes debido a la mala elección de pivote. Finalmente, `Sort` obtiene los mejores tiempos en todos los casos, gracias a su implementación híbrida que evita el peor caso.

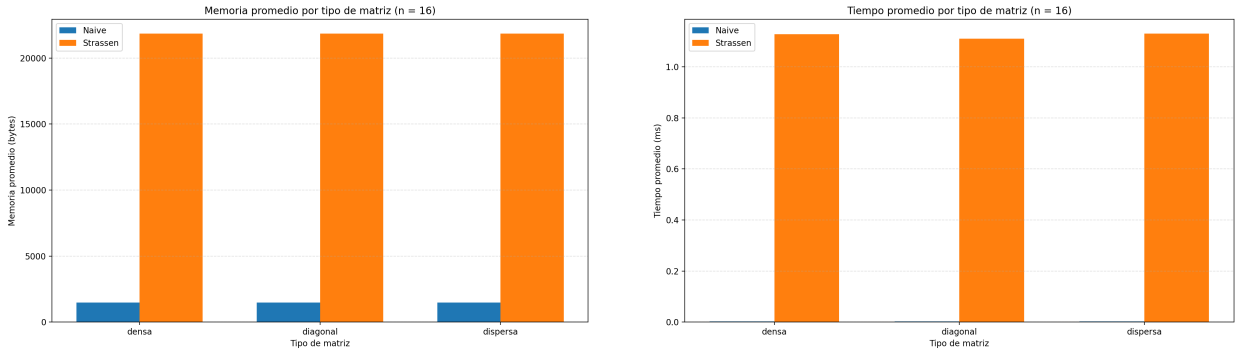


(a) Memoria (b) Tiempos

Figura 3: Resultados para n=10^5

Para $n = 10^5$, el comportamiento de la memoria se mantiene igual que en los casos anteriores. Sin embargo, destaca el bajo rendimiento de QuickSort en los casos ascendente y descendente, cuyos tiempos crecen de tal forma que dejan fuera de escala los de MergeSort y `sort` en la gráfica.
En un análisis general, se observa que QuickSort presenta un mal rendimiento cuando el arreglo está ordenado de forma ascendente o descendente, ya que en estos casos su complejidad se degrada a $O(n^2)$, lo que se acentúa al aumentar n. Por otro lado, `sort` mantiene un buen desempeño en todos los casos gracias a su implementación híbrida, que evita el peor caso de QuickSort. Finalmente, MergeSort presenta tiempos estables independientes del tipo de entrada, aunque su principal desventaja es el uso de memoria adicional proporcional al tamaño del arreglo.

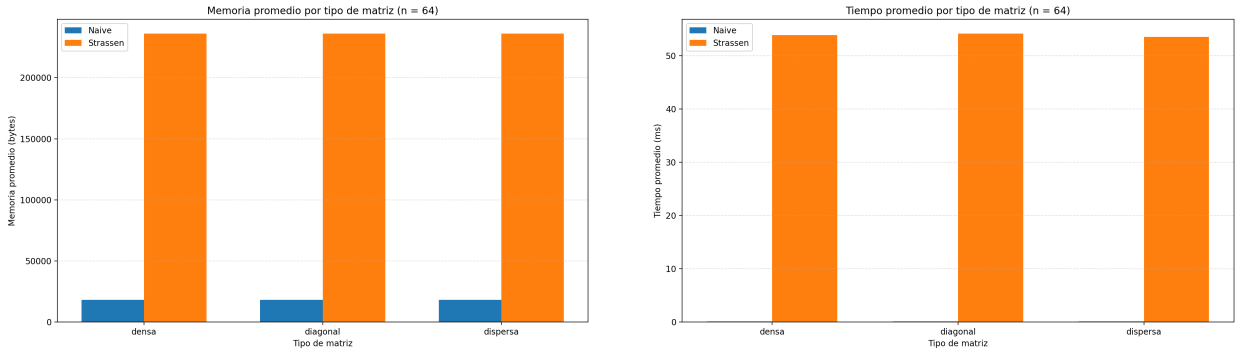
A continuación, se presenta el análisis de los algoritmos de multiplicación de matrices.



(a) Memoria (b) Tiempos

Figura 4: Resultados para matrices 16x16

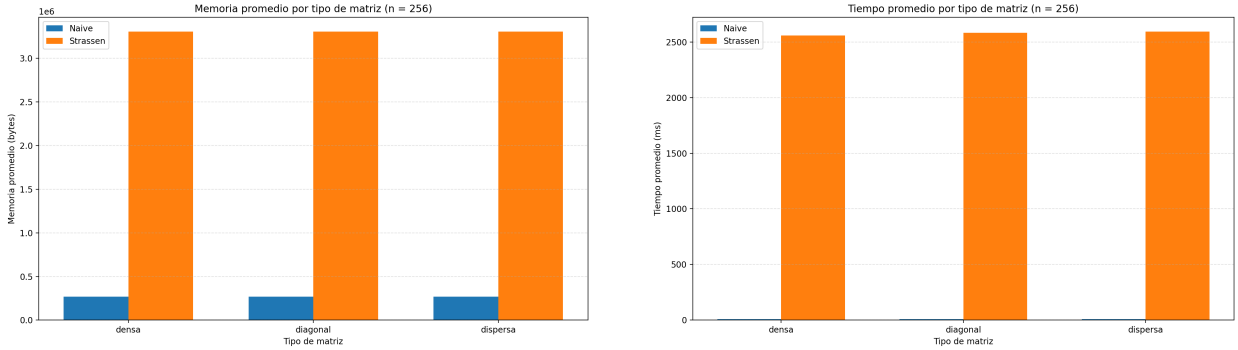
Para $n = 16$, se observa que Strassen utiliza considerablemente más memoria que el algoritmo naive en todos los tipos de matrices. En cuanto al tiempo de ejecución, el algoritmo naive es claramente más rápido, mientras que Strassen presenta un mayor costo debido al overhead de su implementación recursiva.



(a) Memoria (b) Tiempos

Figura 5: Resultados para matrices 64x64

Para $n = 64$, se mantiene la misma tendencia observada anteriormente. Los gráficos presentan proporciones similares, aunque cambia la escala, ya que tanto el uso de memoria como los tiempos de ejecución aumentan para ambos algoritmos.



(a) Memoria (b) Tiempos

Figura 6: Resultados para matrices 256x256

Para $n = 256$, el uso de memoria aumenta considerablemente, lo que provoca un cambio en la escala del gráfico para poder representar ambas implementaciones. En cuanto al tiempo de ejecución, el algoritmo naive incrementa ligeramente su tiempo respecto a casos anteriores; sin embargo, debido a la gran diferencia con Strassen, este aumento no se aprecia claramente en la gráfica.

Para interpretar estos resultados, es importante destacar que el algoritmo de Strassen tuvo un mal desempeño debido a la en esta igual implementación. A pesar de que teóricamente presenta una mejor complejidad que el método naive, en la práctica su rendimiento depende fuertemente de factores como los costos constantes y el manejo de memoria. En particular, la implementación utilizada introduce un alto overhead debido a que las matrices se pasan por valor en las llamadas recursivas, generando copias completas en cada nivel. Además, el algoritmo divide las matrices en múltiples submatrices y crea estructuras auxiliares para las operaciones intermedias, lo que incrementa significativamente el uso de memoria dinámica. Este comportamiento es consistente con lo indica la referencia utilizada para la implementación del algoritmo, donde se menciona que el método de Strassen suele presentar altos costos constantes y requerimientos adicionales de memoria, lo que puede hacerlo menos eficiente que el algoritmo naive en tamaños moderados. Por esta razón, en los experimentos realizados, la sobrecarga de implementación domina sobre la mejora teórica, resultando en un peor desempeño tanto en tiempo como en uso de memoria.

3. Conclusiones

De este trabajo se puede concluir que la implementación de los algoritmos influye directamente en su comportamiento, generando diferencias entre el rendimiento teórico y el observado experimentalmente. Esto fue especialmente notorio en el caso del algoritmo de Strassen, donde la creación de copias y el uso intensivo de recursión provocan un alto consumo de memoria y un tiempo de ejecución mayor al esperado. Asimismo, se observó que para valores pequeños de n todos los algoritmos presentan un desempeño aceptable. Sin embargo, a medida que $n$crece, las diferencias entre ellos se vuelven más evidentes, lo que resalta la importancia de considerar tanto la complejidad teórica como la implementación práctica al evaluar algoritmos.